PRODUCT REQUIREMENTS DOCUMENT

Millets Momo — Daily Order Tracking PWA

Monorepo · React + MUI · Node.js (Azure Functions) · Azure SQL Database

Version 1.0
Document Date: 13 June 2026

> *AUDIENCE: This document is written for an autonomous AI coding agent. It is fully self-contained, specifies exact implementation details (schema, API contracts, design tokens, file structure, security rules, and deployment steps), and requires no further clarification from a human before implementation begins.*

Table of Contents

1. Project Overview

Millets Momo is a small food cart business selling momos (steamed/fried dumplings) in 24 fixed combinations (4 fillings × 6 preparation styles). This project delivers a Progressive Web App (PWA) used internally by the business to record customer orders at the cart and to view daily sales summaries.

The application has two personas:

- Staff (cart worker): logs in with a PIN, selects a date, and records orders as customers place them, with item selection, quantity, half-plate option, order type (Dine in / Pack), and payment status (Cash / UPI / Pending).
- Admin (owner): logs in with a separate PIN and views a daily dashboard — total orders, total revenue, payment-method breakdown, pending amount, and a per-item quantity/revenue breakdown for any selected date.

The application must be installable as a PWA directly from the browser on both iOS (Safari "Add to Home Screen") and Android (Chrome "Install app"), and must work well on phone, tablet, and desktop screens. It will be used by no more than 10 concurrent users at any time, so the architecture favours simplicity, low operating cost (Azure free tiers), and low maintenance over horizontal scalability.

A fully working HTML/CSS/JS prototype of the UI already exists and defines the exact visual design, colour palette, spacing, component layout, copy/wording, and interaction patterns (PIN pad, menu grid, order cards, completion flow, admin dashboard). This PRD converts that prototype into a production-grade monorepo application while preserving its design 1:1. Section 6 ("Design System & Theming") translates every visual detail of the prototype into an MUI theme specification — the coding agent MUST reproduce this design exactly, not redesign it.

2. Goals, Scope & Non-Goals

2.1 In-Scope Functional Requirements

1. PIN-based login with two roles: Staff and Admin, selectable via tabs labelled "Staff" and "Admin" on the login screen.
2. Date selection screen (Staff): pick any date (defaults to today), with "Today" / "Yesterday" quick-select buttons and a list of the 5 most recent days with order counts and revenue totals.
3. Day View (Staff): shows all orders placed for the selected date, with summary stat chips (order count, revenue, pending amount), an "Active orders" section, a horizontal divider, and a "Completed" section below it.
4. "New Order" creation flow reachable via a floating action button and a top-bar button.
5. Menu grid for order entry: a 6×4 grid (6 preparation styles × 4 fillings = 24 items). Tapping a cell adds 1 unit of that item to the order; tapping again increases quantity. Long-press (or right-click on desktop) toggles Half Plate for that item. No prices are shown per cell except a small full-price hint; the live order total is shown in a sticky bottom bar.
6. Order summary panel below the menu and order-detail controls, listing each selected item with quantity +/- controls, a Full/Half toggle, and a line total. Order name format is "{Filling} {Preparation}", e.g. "Veg Steam", "Paneer Creamy Fry", "Cheese Corn Pan Fried".

7. Order detail controls: Order Type — "Dine in" or "Pack" (default Dine in); Payment Method — "Cash", "UPI", or "Pending" (default Cash).
8. On submit, the order is saved with a timestamp-based order number/ID and a human-readable time label (e.g. "02:45 PM", Asia/Kolkata time), and appears as a new order card in the Day View "Active orders" section.
9. Each order card shows: time label, order-type badge, payment badge, item list with quantities and half-plate indicators, total amount, a "✓ Done" button, and a "✕" delete button.
10. Marking an order "Done": if its payment method is "Pending", a bottom-sheet modal must appear asking the staff member to choose the actual payment method received (Cash or UPI) before the order is marked complete and its payment_method is updated accordingly. If payment is already Cash or UPI, the order is marked complete immediately.
11. Completed orders move to the "Completed" section below the divider on the Day View, visually de-emphasised (reduced opacity, grey left border).
12. Admin Dashboard: a separate tab/screen reachable only after Admin login. Contains a date picker, four summary cards (Total orders, Total revenue, Pending amount, Cash/UPI split), a per-item breakdown table (item name, total quantity sold, total revenue) sorted by quantity descending with a totals row, and a read-only list of all orders for that date.
13. Responsive layout for phone (≥360px), tablet (≥768px), and desktop (≥1024px), matching the prototype's breakpoints and behaviour.
14. Smooth, subtle animations matching the prototype: fade-in on screen transitions, slide-up on new order cards and modals, pop-in on the login card, shake on PIN error.
15. Basic validation: cannot submit an order with zero items; cannot proceed without selecting a date; PIN must be exactly 4 digits; date inputs must be valid dates.

2.2 Non-Functional Requirements (Summary)

- Installable PWA on iOS Safari and Android Chrome, working fully offline for the app shell.
- Security: PIN hashing, JWT-based session, Helmet security headers, parameterized SQL queries (no SQL injection), input validation (no XSS), rate limiting on login.
- Unit tests on both frontend and backend covering business logic, components, and API endpoints.
- Well-documented, well-structured TypeScript code with a complete root README.md.
- Deployable on Azure free tiers: Azure Static Web Apps (frontend + managed Functions API) and Azure SQL Database (Free offer tier).

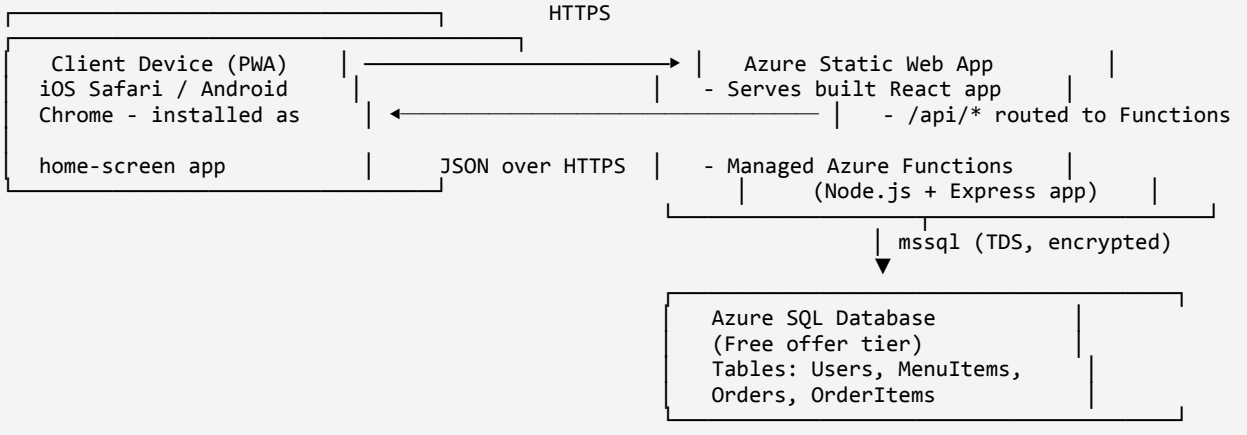
2.3 Out of Scope (Explicitly Not Required)

- Editing an order after submission (only "mark complete" and "delete" are supported).
- Customer-facing ordering (this is an internal staff/admin tool only).
- Online/integrated payment processing (Cash/UPI are recorded as labels only, no payment gateway).
- Multi-cart / multi-location support, push notifications, SMS/email, multi-language support.
- User self-registration — only two fixed accounts (Staff, Admin) seeded at setup time.

3. System Architecture

3.1 High-Level Architecture

The system is a single-page application (SPA) built with React, served as static files together with a Service Worker (PWA). All dynamic data flows through a REST API implemented as a single Azure Functions app (Node.js + Express, wrapped via an HTTP trigger). The API connects to a single Azure SQL Database (Free offer tier) using parameterized queries via the "mssql" package.



3.2 Authentication Flow

16. Staff/Admin selects a tab ("Staff" or "Admin") on the login screen and enters a 4-digit PIN via an on-screen numeric pad.
17. Frontend POSTs { role, pin } to POST /api/auth/login.
18. Backend looks up the active user(s) for that role, compares the PIN against the stored bcrypt hash using bcrypt.compare.
19. On success, backend issues a signed JWT (containing userId, role, displayName, exp) and returns it with role and displayName.
20. Frontend stores the JWT in memory + sessionStorage (not localStorage, to reduce persistence risk on shared devices), and attaches it as "Authorization: Bearer <token>" on all subsequent API calls.
21. Backend middleware verifies the JWT on every protected route and enforces role-based access (e.g., only "admin" may call /api/admin/summary).
22. On 401 responses, the frontend clears the session and redirects to the login screen.

3.3 Data Flow for Orders

- Staff opens Day View for a date → frontend calls GET /api/orders?date=YYYY-MM-DD → backend returns all orders + nested items for that date, split by completed status (or frontend splits client-side).
- Staff builds a new order in the New Order screen (client-side state only) → on submit, frontend calls POST /api/orders with the full order payload (items, totals, type, payment) → backend validates, computes/verifies totals server-side, inserts Orders + OrderItems rows in a transaction, returns the created order.
- Staff marks an order complete → frontend calls PATCH /api/orders/:id/complete with an optional { paymentMethod } body (required only if the order's current payment_method is

"pending") → backend updates is_completed, completed_at, and payment_method if provided.

- Staff deletes an order → DELETE /api/orders/:id → backend deletes the order (cascade deletes its items).
- Admin opens dashboard for a date → GET /api/admin/summary?date=YYYY-MM-DD → backend aggregates totals, payment splits, and per-item breakdown in SQL and returns a single summary object plus the order list.

4. Technology Stack

Layer	Technology	Notes
Frontend framework	React 18 + TypeScript	Functional components + hooks only
Build tool	Vite	Fast dev server, small production bundles
UI library	MUI (Material UI) v5	Theme customised per Section 6
PWA tooling	vite-plugin-pwa (Workbox)	Manifest + service worker + offline cache
Routing	react-router-dom v6	Client-side routing with protected routes
Server state	@tanstack/react-query	Caching, refetch, optimistic updates
HTTP client	axios	Centralised API client with interceptors
Backend runtime	Node.js 20 LTS	Matches Azure Functions Node 20 support
Backend framework	Express.js + TypeScript	Wrapped by Azure Functions HTTP trigger via serverless-http
Hosting platform	Azure Functions v4 (Node programming model)	Bundled inside Azure Static Web Apps "api" location
Database	Azure SQL Database (Free offer tier)	Serverless compute, T-SQL, accessed via "mssql" package
Authentication	JWT (jsonwebtoken) + bcrypt	PINs hashed with bcrypt; sessions via short-lived JWT
Security middleware	helmet, cors, express-rate-limit	Applied in Express app (see Section 13)
Validation	zod	Schema validation for all request bodies/queries
Frontend testing	Vitest + React Testing Library	Component & utility unit tests
Backend testing	Jest + Supertest + ts-jest	Controller, middleware, and integration tests
Linting/formatting	ESLint + Prettier	Shared config at repo root
Hosting/Deployment	Azure Static Web Apps + GitHub Actions	See Section 15

5. Monorepo Structure

The project MUST be a single Git repository using npm workspaces, structured exactly as follows. Create every file/folder listed, even if some files only contain scaffolding initially.

```
millets-momo/
├── apps/
│   ├── frontend/                      # React + Vite + MUI PWA
│   │   ├── public/
│   │   │   ├── icons/                 # PWA icons: 64,128,192,256,384,512 px (PNG)
│   │   │   │   └── maskable-512.png
│   │   │   └── robots.txt
│   │   ├── src/
│   │   │   ├── api/
│   │   │   │   ├── client.ts          # axios instance + interceptors
│   │   │   │   ├── authApi.ts
│   │   │   │   ├── menuApi.ts
│   │   │   │   ├── ordersApi.ts
│   │   │   │   └── adminApi.ts
│   │   │   ├── components/
│   │   │   │   ├── PinPad.tsx
│   │   │   │   ├── MenuGrid.tsx
│   │   │   │   ├── OrderConfigPanel.tsx
│   │   │   │   ├── SelectedItemsList.tsx
│   │   │   │   ├── TotalBar.tsx
│   │   │   │   ├── OrderCard.tsx
│   │   │   │   ├── PaymentModal.tsx
│   │   │   │   ├── StatChip.tsx
│   │   │   │   ├── Toast.tsx
│   │   │   │   └── ProtectedRoute.tsx
│   │   │   ├── pages/
│   │   │   │   ├── LoginPage.tsx
│   │   │   │   ├── DateSelectPage.tsx
│   │   │   │   ├── DayViewPage.tsx
│   │   │   │   ├── NewOrderPage.tsx
│   │   │   │   └── AdminDashboardPage.tsx
│   │   │   ├── context/
│   │   │   │   ├── AuthContext.tsx
│   │   │   │   └── OrderDraftContext.tsx
│   │   │   ├── hooks/
│   │   │   │   ├── useOrders.ts
│   │   │   │   ├── useMenu.ts
│   │   │   │   └── useAdminSummary.ts
│   │   │   ├── theme/
│   │   │   │   ├── theme.ts           # MUI theme (Section 6)
│   │   │   │   └── tokens.ts          # status color tokens
│   │   │   ├── utils/
│   │   │   │   ├── pricing.ts         # half-price formula, totals
│   │   │   │   ├── dateUtils.ts
│   │   │   │   └── formatters.ts
│   │   │   ├── types/
│   │   │   │   └── index.ts           # shared TS interfaces (mirrors packages/shared)
│   │   │   ├── App.tsx
│   │   │   ├── main.tsx
│   │   │   └── vite-env.d.ts
│   │   ├── tests/
│   │   │   ├── PinPad.test.tsx
│   │   │   ├── MenuGrid.test.tsx
│   │   │   ├── TotalBar.test.tsx
│   │   │   ├── OrderCard.test.tsx
│   │   │   ├── PaymentModal.test.tsx
│   │   │   └── pricing.test.ts
│   │   ├── index.html
│   │   ├── vite.config.ts
│   │   ├── tsconfig.json
│   │   ├── .eslintrc.cjs
```

```
│   ├── .env.example
│   └── package.json
│
└── backend/                        # Azure Functions (Node.js + Express)
    ├── src/
    │   ├── functions/
    │   │   └── api.ts              # single HTTP trigger, wraps Express app
    │   ├── app.ts                  # Express app: middleware + route mounting
    │   ├── routes/
    │   │   ├── authRoutes.ts
    │   │   ├── menuRoutes.ts
    │   │   ├── ordersRoutes.ts
    │   │   └── adminRoutes.ts
    │   ├── controllers/
    │   │   ├── authController.ts
    │   │   ├── menuController.ts
    │   │   ├── ordersController.ts
    │   │   └── adminController.ts
    │   ├── services/
    │   │   ├── authService.ts
    │   │   ├── menuService.ts
    │   │   ├── ordersService.ts
    │   │   └── adminService.ts
    │   ├── db/
    │   │   ├── pool.ts             # mssql connection pool singleton
    │   │   ├── schema.sql          # DDL (Section 8)
    │   │   └── seed.sql            # seed data (Sections 8 & 9)
    │   ├── middleware/
    │   │   ├── authMiddleware.ts
    │   │   ├── errorHandler.ts
    │   │   └── rateLimiter.ts
    │   ├── validators/
    │   │   ├── authValidators.ts
    │   │   └── orderValidators.ts
    │   ├── constants/
    │   │   └── menu.ts             # 24-item menu + pricing (Section 9)
    │   └── utils/
    │       ├── pricing.ts          # half-price formula (shared logic)
    │       └── time.ts             # Asia/Kolkata date/time helpers
    ├── tests/
    │   ├── auth.test.ts
    │   ├── menu.test.ts
    │   ├── orders.test.ts
    │   ├── admin.test.ts
    │   └── pricing.test.ts
    ├── scripts/
    │   ├── migrate.ts              # runs schema.sql against Azure SQL
    │   └── generatePinHash.ts      # CLI: bcrypt-hash a PIN for env vars
    ├── host.json
    ├── local.settings.example.json
    ├── tsconfig.json
    ├── .eslintrc.cjs
    └── package.json

├── packages/
│   └── shared/                     # Types & constants shared by both apps
│       ├── src/
│       │   ├── types.ts            # Order, OrderItem, MenuItem, User, etc.
│       │   ├── menu.ts             # canonical 24-item menu definition
│       │   └── pricing.ts          # canonical half-price formula
│       ├── package.json
│       └── tsconfig.json
│
├── .github/
│   └── workflows/
│       └── azure-deploy.yml
├── docs/
```

```
│   ├── ARCHITECTURE.md
│   ├── API.md
│   └── DEPLOYMENT.md
├── .gitignore
├── .editorconfig
├── package.json                        # root workspaces config
└── README.md
```

6. Design System & Theming

The existing HTML prototype defines the exact visual identity of the app. The coding agent MUST reproduce these design tokens precisely via an MUI theme — do not invent new colours, radii, or spacing. Implement this as apps/frontend/src/theme/theme.ts and apps/frontend/src/theme/tokens.ts.

6.1 Colour Palette

Token	Hex	Usage
Primary (green)	#1B6B3A	Top bar, primary buttons, FAB, active states
Primary light	#E8F5EE	Selected menu cells, login icon background, chips
Primary mid	#2D8A4E	Secondary accents, stat values
Primary dark	#124D29	Pressed/active button states, gradients
Amber	#D97706	Pack badge, Pending payment accents
Amber light	#FEF3C7	Pack badge background
Red	#DC2626	Pending payment badge, delete buttons, errors
Red light	#FEE2E2	Pending badge background, error backgrounds
Blue	#1D4ED8	Dine-in badge text
Blue light	#EFF6FF	Dine-in badge background
Page background	#F0F4F1	Overall app background
Surface (paper)	#FFFFFF	Cards, modals, top-of-stack surfaces
Grey 50–900	#F9FAFB → #111827	Standard MUI grey scale, see 6.2

6.2 Grey Scale

```
grey: {
  50:  '#F9FAFB', 100: '#F3F4F6', 200: '#E5E7EB', 300: '#D1D5DB',
  400: '#9CA3AF', 500: '#6B7280', 600: '#4B5563', 700: '#374151',
  800: '#1F2937', 900: '#111827',
}
```

6.3 Status / Badge Colour Tokens

Define these in apps/frontend/src/theme/tokens.ts and use them for order-card badges and admin summary cards:

```
export const statusColors = {
```

```
  dineIn:    { bg: '#EFF6FF', fg: '#1D4ED8', label: '🍽️ Dine in' },
  pack:      { bg: '#FEF3C7', fg: '#D97706', label: '📦 Pack' },
  cash:      { bg: '#D1FAE5', fg: '#065F46', label: '💵 Cash' },
  upi:       { bg: '#EDE9FE', fg: '#5B21B6', label: '📱 UPI' },
  pending:   { bg: '#FEE2E2', fg: '#DC2626', label: '⏳ Pending' },
  completed: { bg: '#F3F4F6', fg: '#6B7280', label: '✓ Done' },
};
```

6.4 Shape, Spacing & Shadows

Token	Value	MUI mapping
Radius — small	8px	Buttons, inputs (shape.borderRadius base unit)
Radius — medium	12px	Stat chips, config buttons, FAB inner elements
Radius — large	16px	Cards (MuiCard override)
Radius — extra large	20px	Login card, modals/bottom sheets
Shadow — small	0 1px 3px rgba(0,0,0,0.08), 0 1px 2px rgba(0,0,0,0.04)	Cards at rest
Shadow — medium	0 4px 12px rgba(0,0,0,0.10), 0 2px 4px rgba(0,0,0,0.06)	FAB, raised elements
Shadow — large	0 10px 30px rgba(0,0,0,0.12), 0 4px 8px rgba(0,0,0,0.08)	Login card, modals

6.5 Typography

- Font stack: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif (do not load a custom web font — keeps the bundle light and matches native rendering on iOS/Android).
- Base font size: 15px body text.
- Headings and totals use font-weight 700–800 with slightly tightened letter-spacing (-0.2px to -0.5px) for a crisp, modern feel, matching the prototype's "Total amount" display.
- Buttons use font-weight 600 and textTransform: "none" (no uppercase — MUI default uppercase must be disabled).

6.6 MUI Theme Implementation

apps/frontend/src/theme/theme.ts must export a theme created via createTheme() with at least the following configuration:

```
import { createTheme } from '@mui/material/styles';

export const theme = createTheme({
  palette: {
    mode: 'light',
    primary: {
```

```
      main: '#1B6B3A',
      light: '#E8F5EE',
      dark: '#124D29',
      contrastText: '#FFFFFF',
    },
    secondary: {
      main: '#D97706',
      light: '#FEF3C7',
      contrastText: '#FFFFFF',
    },
    error:   { main: '#DC2626', light: '#FEE2E2' },
    info:    { main: '#1D4ED8', light: '#EFF6FF' },
    success: { main: '#2D8A4E', light: '#D1FAE5' },
    background: { default: '#F0F4F1', paper: '#FFFFFF' },
    grey: {
      50: '#F9FAFB', 100: '#F3F4F6', 200: '#E5E7EB', 300: '#D1D5DB',
      400: '#9CA3AF', 500: '#6B7280', 600: '#4B5563', 700: '#374151',
      800: '#1F2937', 900: '#111827',
    },
    text: { primary: '#111827', secondary: '#6B7280' },
  },
  typography: {
    fontFamily: [
      '-apple-system', 'BlinkMacSystemFont', '"Segoe UI"', 'Roboto', 'sans-serif',
    ].join(','),
    fontSize: 14,
    button: { fontWeight: 600, textTransform: 'none' },
    h1: { fontWeight: 800, letterSpacing: '-0.5px' },
    h2: { fontWeight: 700, letterSpacing: '-0.3px' },
  },
  shape: { borderRadius: 12 },
  components: {
    MuiButton: {
      styleOverrides: {
        root: { borderRadius: 8, fontWeight: 600 },
        containedPrimary: {
          '&:active': { backgroundColor: '#124D29', transform: 'scale(0.97)' },
        },
      },
    },
    MuiCard: {
      styleOverrides: {
        root: {
          borderRadius: 16,
          boxShadow: '0 1px 3px rgba(0,0,0,0.08), 0 1px 2px rgba(0,0,0,0.04)',
        },
      },
    },
    MuiChip: {
      styleOverrides: { root: { borderRadius: 20, fontWeight: 600 } },
    },
    MuiPaper: {
      styleOverrides: { root: { backgroundImage: 'none' } },
    },
  },
});
```

6.7 Animation Requirements

Element	Animation	Approx. duration
Screen/route transitions	Fade + slight upward translate on mount	250ms ease

Login card	Pop-in (scale 0.92 → 1, slight overshoot)	350ms cubic-bezier(0.34,1.56,0.64,1)
PIN error	Horizontal shake on the PIN dot row	400ms ease
New order card appearing in Day View	Slide-up + fade	250ms ease
Payment modal / bottom sheet	Slide up from bottom + dim overlay fade-in	250ms ease
Button / chip press	Scale down to ~0.93–0.97 on press	120ms

> ⚠ *Implement animations with MUI's built-in transition components (Fade, Slide, Grow, Collapse) and CSS keyframes via the theme's sx prop / styled API — avoid adding a separate animation library to keep the bundle light.*

7. PWA Requirements

7.1 Web App Manifest

Configure via vite-plugin-pwa in apps/frontend/vite.config.ts. The generated manifest.webmanifest must include:

```
{
  "name": "Millets Momo - Order Tracker",
  "short_name": "Millets Momo",
  "description": "Daily order tracking for the Millets Momo cart",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "orientation": "portrait",
  "background_color": "#F0F4F1",
  "theme_color": "#1B6B3A",
  "icons": [
    { "src": "/icons/icon-192.png", "sizes": "192x192", "type": "image/png", "purpose": "any" },
    { "src": "/icons/icon-512.png", "sizes": "512x512", "type": "image/png", "purpose": "any" },
    { "src": "/icons/maskable-512.png", "sizes": "512x512", "type": "image/png", "purpose":
"maskable" }
  ]
}
```

7.2 iOS-Specific Requirements

iOS Safari does not fully respect the web manifest, so apps/frontend/index.html MUST also include:

```
<meta name="apple-mobile-web-app-capable" content="yes" />
<meta name="apple-mobile-web-app-status-bar-style" content="default" />
<meta name="apple-mobile-web-app-title" content="Millets Momo" />
<link rel="apple-touch-icon" href="/icons/icon-192.png" />
<meta name="theme-color" content="#1B6B3A" />
```

7.3 Service Worker / Offline Behaviour

- Use vite-plugin-pwa with strategy: "generateSW" and registerType: "autoUpdate".
- Precache the app shell (HTML, JS, CSS, icons, fonts) so the login screen and static UI load offline.

- Runtime caching: GET /api/menu uses a "CacheFirst" strategy with a long expiry (menu rarely changes); GET /api/orders and GET /api/admin/summary use "NetworkFirst" with a short cache fallback so the UI can still render the last-seen data if briefly offline.
- Do not cache POST/PATCH/DELETE requests (orders must not be created/modified while offline, since this is a single-source-of-truth order system — show a clear "You are offline, please reconnect" message if a mutation fails due to network error).

7.4 Installability Checklist

23. App is served over HTTPS (provided automatically by Azure Static Web Apps).
24. Valid manifest with name, icons (192px and 512px minimum), start_url, and display: "standalone".
25. Registered service worker with a fetch handler.
26. Verified installable via Lighthouse PWA audit (target score ≥ 90) and manually tested: "Add to Home Screen" on iOS Safari, and the install prompt / "Install app" menu item on Android Chrome.

8. Data Model & Database Schema (Azure SQL / T-SQL)

Save the following as apps/backend/src/db/schema.sql. All queries against these tables MUST use parameterized statements via the "mssql" package (see Section 13.4) — never string concatenation.

8.1 Users Table

```
CREATE TABLE Users (
  id           INT IDENTITY(1,1) PRIMARY KEY,
  username     NVARCHAR(50)  NOT NULL UNIQUE,
  role         NVARCHAR(20)  NOT NULL CHECK (role IN ('staff','admin')),
  pin_hash     NVARCHAR(255) NOT NULL,
  display_name NVARCHAR(100) NOT NULL,
  is_active    BIT           NOT NULL DEFAULT 1,
  created_at   DATETIME2     NOT NULL DEFAULT SYSUTCDATETIME()
);
```

8.2 MenuItems Table

```
CREATE TABLE MenuItems (
  id           INT IDENTITY(1,1) PRIMARY KEY,
  filling      NVARCHAR(30)  NOT NULL,
  preparation  NVARCHAR(30)  NOT NULL,
  display_name NVARCHAR(60)  NOT NULL,
  full_price   DECIMAL(6,2)  NOT NULL,
  half_price   DECIMAL(6,2)  NOT NULL,
  is_active    BIT           NOT NULL DEFAULT 1,
  CONSTRAINT UQ_MenuItems_Combo UNIQUE (filling, preparation)
);
```

8.3 Orders Table

```
CREATE TABLE Orders (
  id              BIGINT        NOT NULL PRIMARY KEY,  -- epoch ms, generated by backend at submit
time
  order_date      DATE          NOT NULL,              -- the selected business date (YYYY-MM-DD)
  time_label      NVARCHAR(20)  NOT NULL,              -- e.g. '02:45 PM' (Asia/Kolkata)
  order_type      NVARCHAR(10)  NOT NULL CHECK (order_type IN ('dine','pack')),
  payment_method  NVARCHAR(10)  NOT NULL CHECK (payment_method IN ('cash','upi','pending')),
```

```
  is_completed    BIT           NOT NULL DEFAULT 0,
  total_amount    DECIMAL(8,2)  NOT NULL,
  created_by      INT           NULL REFERENCES Users(id),
  created_at      DATETIME2     NOT NULL DEFAULT SYSUTCDATETIME(),
  completed_at    DATETIME2     NULL
);

CREATE INDEX IX_Orders_OrderDate  ON Orders(order_date);
CREATE INDEX IX_Orders_Completed  ON Orders(order_date, is_completed);
```

8.4 OrderItems Table

```
CREATE TABLE OrderItems (
  id            INT     IDENTITY(1,1) PRIMARY KEY,
  order_id      BIGINT  NOT NULL REFERENCES Orders(id) ON DELETE CASCADE,
  menu_item_id  INT     NOT NULL REFERENCES MenuItems(id),
  item_name     NVARCHAR(60)  NOT NULL,  -- denormalised snapshot, e.g. 'Veg Steam'
  quantity      INT     NOT NULL CHECK (quantity > 0),
  is_half       BIT     NOT NULL DEFAULT 0,
  unit_price    DECIMAL(6,2)  NOT NULL,  -- price actually charged (full or half)
  line_total    DECIMAL(8,2)  NOT NULL  -- unit_price * quantity
);

CREATE INDEX IX_OrderItems_OrderId ON OrderItems(order_id);
```

> ⚠ *Why item_name and unit_price are denormalised onto OrderItems: menu prices may change in the future, but historical orders must always reflect the price actually charged on the day. Never recompute historical totals from the live MenuItems table.*

9. Menu & Pricing Reference Data

The menu consists of exactly 24 fixed items: 6 preparation styles × 4 fillings. Display name = "{Filling} {Preparation}" (e.g. "Veg Steam", "Platter Pan Fried"). All prices below are taken from the business's printed menu card and were verified with the business owner. This canonical definition MUST live in packages/shared/src/menu.ts (and be re-exported by both frontend and backend) AND be seeded into the MenuItems table via seed.sql.

9.1 Full-Plate Price Grid (₹)

Preparation \ Filling	Veg	Paneer	Cheese Corn	Platter
Steam	89	109	129	109
Fry	109	129	149	129
Creamy	129	129	149	129
Creamy Fry	129	149	169	149
Nep. Kothey	129	139	149	139
Pan Fried	139	149	159	149

9.2 Half-Plate Price Grid (₹)

Preparation \ Filling	Veg	Paneer	Cheese Corn	Platter
Steam	50	60	70	60
Fry	60	70	80	70

Creamy	60	70	80	70
Creamy Fry	70	80	90	80
Nep. Kothey	70	75	80	75
Pan Fried	75	80	85	80

Sourcing notes for the half-plate grid:

- Steam, Fry, Creamy, and Creamy Fry half prices are printed directly on the menu card for Veg, Paneer, and Cheese Corn; the Platter column for these four rows is copied from the Paneer column (confirmed with the business — Platter is priced the same as Paneer where not separately listed).
- Nep. Kothey and Pan Fried have no printed half-plate price. These were computed using the formula round((fullPrice + 11) / 2), confirmed with the business as the standard rule for any item without an explicit half price.

9.3 Half-Plate Pricing Formula (Fallback Rule)

For any future menu item that is added without an explicit half price, apply:

```
halfPrice = Math.round((fullPrice + 11) / 2)
```

This formula is implemented in packages/shared/src/pricing.ts as a utility (computeHalfPrice), but it is NOT used to derive the 24 stored values in Section 9.2 — those are fixed, verified values seeded directly into MenuItems.half_price. The formula exists purely as a documented fallback rule for future menu additions. Example values:

Full Price (₹)	Half Price via formula (₹)	Calculation
89	50	round((89+11)/2) = 50
109	60	round((109+11)/2) = 60
129	70	round((129+11)/2) = 70
139	75	round((139+11)/2) = 75
149	80	round((149+11)/2) = 80
159	85	round((159+11)/2) = 85
169	90	round((169+11)/2) = 90
179	95	round((179+11)/2) = 95
189	100	round((189+11)/2) = 100
199	105	round((199+11)/2) = 105

9.4 Canonical Menu Definition (TypeScript)

Implement exactly as follows in packages/shared/src/menu.ts:

```
export const FILLINGS = ['Veg', 'Paneer', 'Cheese Corn', 'Platter'] as const;
export const PREPARATIONS = [
  'Steam', 'Fry', 'Creamy', 'Creamy Fry', 'Nep. Kothey', 'Pan Fried',
] as const;

// FULL_PRICES[prepIndex][fillingIndex] = full plate price in INR
```

```
// Source: printed Millets Momo menu card (verified with business owner).
// Platter (Steam/Fry/Creamy/Creamy Fry) is priced the same as Paneer.
export const FULL_PRICES: number[][] = [
  [89, 109, 129, 109],   // Steam
  [109, 129, 149, 129],  // Fry
  [129, 129, 149, 129],  // Creamy
  [129, 149, 169, 149],  // Creamy Fry
  [129, 139, 149, 139],  // Nep. Kothey
  [139, 149, 159, 149],  // Pan Fried
];

// HALF_PRICES[prepIndex][fillingIndex] = half plate price in INR
// Steam/Fry/Creamy/Creamy Fry: printed on menu (Platter copied from Paneer).
// Nep. Kothey / Pan Fried: computed via round((full + 11) / 2) — no printed value.
export const HALF_PRICES: number[][] = [
  [50, 60, 70, 60],   // Steam
  [60, 70, 80, 70],   // Fry
  [60, 70, 80, 70],   // Creamy
  [70, 80, 90, 80],   // Creamy Fry
  [70, 75, 80, 75],   // Nep. Kothey
  [75, 80, 85, 80],   // Pan Fried
];

export interface MenuItem {
  filling: string;
  preparation: string;
  displayName: string;   // "{Filling} {Preparation}"
  fullPrice: number;
  halfPrice: number;
}

export function buildMenu(): MenuItem[] {
  const items: MenuItem[] = [];
  PREPARATIONS.forEach((prep, pi) => {
    FILLINGS.forEach((fill, fi) => {
      items.push({
        filling: fill,
        preparation: prep,
        displayName: `${fill} ${prep}`,
        fullPrice: FULL_PRICES[pi][fi],
        halfPrice: HALF_PRICES[pi][fi],
      });
    });
  });
  return items; // 24 items
}

/** Fallback rule for any future item with no explicit half price (Section 9.3). */
export function computeHalfPrice(fullPrice: number): number {
  return Math.round((fullPrice + 11) / 2);
}
```

9.5 Seed Data (seed.sql)

Append to apps/backend/src/db/seed.sql — one INSERT per menu item (24 rows total), generated from the grids above:

```
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Veg',
'Steam', 'Veg Steam', 89.00, 50.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Paneer',
'Steam', 'Paneer Steam', 109.00, 60.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Cheese
Corn', 'Steam', 'Cheese Corn Steam', 129.00, 70.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES
('Platter', 'Steam', 'Platter Steam', 109.00, 60.00);
```

```
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Veg',
'Fry', 'Veg Fry', 109.00, 60.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Paneer',
'Fry', 'Paneer Fry', 129.00, 70.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Cheese
Corn', 'Fry', 'Cheese Corn Fry', 149.00, 80.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES
('Platter', 'Fry', 'Platter Fry', 129.00, 70.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Veg',
'Creamy', 'Veg Creamy', 129.00, 60.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Paneer',
'Creamy', 'Paneer Creamy', 129.00, 70.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Cheese
Corn', 'Creamy', 'Cheese Corn Creamy', 149.00, 80.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES
('Platter', 'Creamy', 'Platter Creamy', 129.00, 70.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Veg',
'Creamy Fry', 'Veg Creamy Fry', 129.00, 70.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Paneer',
'Creamy Fry', 'Paneer Creamy Fry', 149.00, 80.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Cheese
Corn', 'Creamy Fry', 'Cheese Corn Creamy Fry', 169.00, 90.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES
('Platter', 'Creamy Fry', 'Platter Creamy Fry', 149.00, 80.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Veg',
'Nep. Kothey', 'Veg Nep. Kothey', 129.00, 70.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Paneer',
'Nep. Kothey', 'Paneer Nep. Kothey', 139.00, 75.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Cheese
Corn', 'Nep. Kothey', 'Cheese Corn Nep. Kothey', 149.00, 80.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES
('Platter', 'Nep. Kothey', 'Platter Nep. Kothey', 139.00, 75.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Veg',
'Pan Fried', 'Veg Pan Fried', 139.00, 75.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Paneer',
'Pan Fried', 'Paneer Pan Fried', 149.00, 80.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES ('Cheese
Corn', 'Pan Fried', 'Cheese Corn Pan Fried', 159.00, 85.00);
INSERT INTO MenuItems (filling, preparation, display_name, full_price, half_price) VALUES
('Platter', 'Pan Fried', 'Platter Pan Fried', 149.00, 80.00);
```

Also seed the two fixed user accounts (PIN hashes generated at setup time via the provided script — see Section 16):

```
-- Run scripts/generatePinHash.ts with the desired 4-digit PIN to obtain a bcrypt hash,
-- then substitute the hashes below before running this seed.
INSERT INTO Users (username, role, pin_hash, display_name) VALUES
  ('staff', 'staff', '<BCRYPT_HASH_OF_STAFF_PIN>', 'Cart Staff'),
  ('admin', 'admin', '<BCRYPT_HASH_OF_ADMIN_PIN>', 'Owner');
```

10. Business Logic & Rules

10.1 Order Identification & Timing

- Order ID: generated by the backend at submission time as Date.now() (epoch milliseconds, BIGINT). This is unique and naturally sortable.
- time_label: formatted as "hh:mm AM/PM" in the Asia/Kolkata timezone, e.g. "02:45 PM", generated server-side at submission time and stored verbatim (do not regenerate on read).

- order_date: the business date the staff member selected on the Date Select screen (not necessarily "today" — staff may back-fill or review past days). Stored as a SQL DATE (YYYY-MM-DD), no time component.

10.2 Order Totals

- Each OrderItem line_total = unitPrice × quantity, where unitPrice is fullPrice or halfPrice from the canonical menu depending on the isHalf flag.
- Order.total_amount = sum of all its OrderItems.line_total.
- The frontend computes and displays the running total live as items are added/adjusted (for instant feedback to quote the customer).
- On POST /api/orders, the backend MUST recompute unitPrice and line_total for every submitted item from the canonical menu (packages/shared/src/menu.ts + pricing.ts) and the recomputed total_amount, IGNORING any totals sent by the client, to prevent tampering. If the recomputed values differ from what the client sent, the backend values win (log a warning but do not reject, since this is an internal trusted-staff tool — but server values are authoritative).

10.3 Order Type & Payment Method

- order_type: "dine" (default, label "🍽️ Dine in") or "pack" (label "📦 Pack").
- payment_method: "cash" (default, label "💵 Cash"), "upi" (label "📱 UPI"), or "pending" (label "⏳ Pending").

10.4 Order Completion & the Pending-Payment Modal

27. Staff taps "✓ Done" on an order card.
28. Frontend checks the order's current payment_method (from local state/cache).
29. If payment_method !== "pending": call PATCH /api/orders/:id/complete with body { } (no paymentMethod). Backend sets is_completed = 1 and completed_at = now.
30. If payment_method === "pending": frontend shows a bottom-sheet PaymentModal titled "Collect Payment" with the message "This order was pending. How did the customer pay?" and two buttons: "💵 Cash" and "📱 UPI", plus a "Cancel" button that dismisses the modal without changes.
31. On choosing Cash or UPI: call PATCH /api/orders/:id/complete with body { paymentMethod: "cash" | "upi" }. Backend updates payment_method to the chosen value, sets is_completed = 1 and completed_at = now, in a single transaction.
32. After completion, the order card moves from the "Active orders" section to the "Completed" section below the divider, with reduced opacity and a grey left border, and a "✓ Done" status badge replaces the action button.

10.5 Day View Grouping & Stats

- Active orders = orders for the date where is_completed = 0, shown first under "Active orders (N)".
- Completed orders = orders where is_completed = 1, shown below a horizontal divider labelled "Completed (N)".
- Stat chips at the top of Day View: "Orders" = total count for the date; "Revenue" = sum of total_amount for ALL orders that date (active + completed); "Pending" = sum of

total_amount for orders where payment_method = "pending" (regardless of completion status).

10.6 Order Deletion

- Deleting an order (DELETE /api/orders/:id) permanently removes the order and its items (cascade). No edit/undo is provided — the frontend MUST show a confirmation dialog ("Remove this order?") before calling the API.

10.7 Validation Rules

Field / Action	Rule
PIN entry	Exactly 4 digits (0-9). Auto-submits when the 4th digit is entered.
Date selection	Must be a valid date; defaults to today (Asia/Kolkata) on first load.
New order submission	At least one menu item with quantity ≥ 1 must be selected. Submit button is disabled otherwise.
Quantity	Integer ≥ 0. Decrementing to 0 removes the item from the order.
Order type	Must be one of "dine" | "pack".
Payment method	Must be one of "cash" | "upi" | "pending".
Complete with pending payment	paymentMethod in the completion request, when required, must be "cash" or "upi" — "pending" is rejected with 400.

11. API Specification

Base path: /api. All endpoints except /api/auth/login and /api/health require a valid JWT in the Authorization header. All request/response bodies are JSON. All list endpoints that take a date use the query parameter "date" in YYYY-MM-DD format.

11.1 Endpoint Summary

Method	Path	Auth	Description
GET	/api/health	None	Liveness check, returns { status: "ok" }
POST	/api/auth/login	None	Authenticate via role + PIN, returns JWT
GET	/api/menu	Staff or Admin	Returns the 24-item menu with full/half prices
GET	/api/orders?date=	Staff or Admin	List all orders (with items) for a date
POST	/api/orders	Staff or Admin	Create a new order
PATCH	/api/orders/:id/complete	Staff or Admin	Mark an order complete (optional payment update)
DELETE	/api/orders/:id	Staff or Admin	Delete an order

GET	/api/admin/summary?date=	Admin only	Daily summary stats + item breakdown

11.2 POST /api/auth/login

Request:

```
{
  "role": "staff" | "admin",
  "pin": "1234"
}
```

Success response (200):

```
{
  "token": "<jwt>",
  "role": "staff",
  "displayName": "Cart Staff",
  "expiresIn": 43200
}
```

Error response (401):

```
{ "error": "Invalid PIN" }
```

> ⚠ *Rate-limited to 5 attempts per 15 minutes per IP (see Section 13.3).*

11.3 GET /api/menu

Success response (200):

```
{
  "items": [
    { "id": 1, "filling": "Veg", "preparation": "Steam", "displayName": "Veg Steam", "fullPrice":
89, "halfPrice": 50 },
    { "id": 2, "filling": "Paneer", "preparation": "Steam", "displayName": "Paneer Steam",
"fullPrice": 109, "halfPrice": 60 }
    // ... 24 items total
  ]
}
```

11.4 GET /api/orders?date=YYYY-MM-DD

Success response (200):

```
{
  "date": "2026-06-13",
  "orders": [
    {
      "id": 1749812345678,
      "orderDate": "2026-06-13",
      "timeLabel": "02:45 PM",
      "orderType": "dine",
      "paymentMethod": "cash",
      "isCompleted": false,
      "totalAmount": 248,
      "items": [
        { "menuItemId": 1, "itemName": "Veg Steam", "quantity": 2, "isHalf": false, "unitPrice": 89,
"lineTotal": 178 },
        { "menuItemId": 9, "itemName": "Paneer Creamy", "quantity": 1, "isHalf": true, "unitPrice":
70, "lineTotal": 70 }
      ]
    }
  ]
}
```

11.5 POST /api/orders

Request:

```
{
  "orderDate": "2026-06-13",
  "orderType": "dine" | "pack",
  "paymentMethod": "cash" | "upi" | "pending",
  "items": [
    { "menuItemId": 1, "quantity": 2, "isHalf": false },
    { "menuItemId": 9, "quantity": 1, "isHalf": true }
  ]
}
```

Validation: items must be a non-empty array; each menuItemId must exist in MenuItems; quantity ≥ 1 integer. Server recomputes unitPrice, lineTotal, and totalAmount from the canonical menu (Section 10.2).

Success response (201): same shape as a single order object in 11.4, including the server-generated id and timeLabel.

11.6 PATCH /api/orders/:id/complete

Request (when current paymentMethod !== "pending"):

```
{}
```

Request (when current paymentMethod === "pending" — paymentMethod is REQUIRED and must be "cash" or "upi"):

```
{ "paymentMethod": "cash" | "upi" }
```

Success response (200): the updated order object. Error (400) if paymentMethod is required but missing/invalid: { "error": "paymentMethod (cash or upi) is required to complete a pending order" }.

11.7 DELETE /api/orders/:id

Success response (200): { "deleted": true, "id": 1749812345678 }. Error (404) if the order does not exist.

11.8 GET /api/admin/summary?date=YYYY-MM-DD

Admin-only. Success response (200):

```
{
  "date": "2026-06-13",
  "totalOrders": 18,
  "totalRevenue": 4280,
  "pendingAmount": 240,
  "cashTotal": 3120,
  "upiTotal": 920,
  "itemBreakdown": [
    { "itemName": "Veg Steam", "totalQuantity": 14, "totalRevenue": 1260 },
    { "itemName": "Paneer Creamy Fry", "totalQuantity": 9, "totalRevenue": 1350 }
    // ... sorted by totalQuantity descending
  ],
  "orders": [ /* same shape as 11.4 orders array, read-only */ ]
}
```

11.9 Error Response Convention

All errors return { "error": "<message>" } with an appropriate HTTP status: 400 (validation), 401 (auth), 403 (wrong role), 404 (not found), 429 (rate limited), 500 (server error, generic message only — no stack traces in production).

12. Frontend Specification

12.1 Routes

Path	Page Component	Access	Description
/login	LoginPage	Public	Role tabs (Staff/Admin) + PIN pad
/dates	DateSelectPage	Staff	Date picker + quick-select + recent days
/day/:date	DayViewPage	Staff	Active/completed order cards for a date
/day/:date/new	NewOrderPage	Staff	Menu grid + order config + summary + submit
/admin	AdminDashboardPage	Admin	Date picker + summary cards + item breakdown + orders list

- ProtectedRoute wraps Staff/Admin routes: checks AuthContext for a valid token + matching role; redirects to /login if absent/invalid; on 401 from any API call, AuthContext clears the session and the router redirects to /login.
- Root "/" redirects to "/login" if unauthenticated, or to "/dates" (staff) / "/admin" (admin) if already authenticated.

12.2 Key Components

Component	Responsibility
PinPad	4-dot PIN display + numeric keypad (1-9, 0, delete, enter); shake animation on wrong PIN; role tabs "Staff"/"Admin" above it.
MenuGrid	6 (preparations) × 4 (fillings) tap-grid. Click = add 1 / increment. Long-press (500ms touch) or right-click (desktop contextmenu) = toggle half-plate for that cell. Shows quantity badge and "½ plate" tag when applicable.
OrderConfigPanel	Two button-groups: Order Type (Dine in / Pack) and Payment (Cash / UPI / Pending), single-select, default Dine in + Cash.
SelectedItemsList	Lists each chosen item with name, Full/Half toggle chip, qty +/- buttons, and line total; shows "No items added yet" when empty.
TotalBar	Sticky bottom bar showing live total (₹) and the "Place order" button (disabled until ≥1 item selected).
OrderCard	Renders one order: time label, type badge, payment badge, item list ("2× Veg Steam", "1× Paneer Creamy (½)"), total, and action buttons ("✓ Done" / "✕" delete) or a "✓ Done" status badge if completed.
PaymentModal	Bottom-sheet modal for resolving a pending payment on completion (Section 10.4).

StatChip	Small rounded card showing a label + value, used for Orders/Revenue/Pending stats and Admin summary cards.
Toast	Transient bottom toast for confirmations ("Order placed! 🎉", "Order removed", "½ plate set", etc.).

12.3 State Management

- AuthContext: holds { token, role, displayName }, persisted to sessionStorage, exposes login()/logout(); axios interceptor reads the token for the Authorization header and triggers logout() on 401.
- OrderDraftContext (or local component state within NewOrderPage): holds the in-progress order — selected items map keyed by menuItemId → { quantity, isHalf }, orderType, paymentMethod. Cleared when navigating away from /day/:date/new (cancel or submit).
- Server state (menu, orders list, admin summary) is managed via @tanstack/react-query: useMenu(), useOrders(date), useAdminSummary(date), with mutations useCreateOrder(), useCompleteOrder(), useDeleteOrder() that invalidate the relevant query on success for automatic UI refresh.

12.4 Responsive Behaviour

Breakpoint	Width	Layout notes
xs (phone)	< 600px	Single column; menu grid scrolls horizontally if needed; bottom sticky TotalBar and FAB; full-width cards.
sm/md (tablet)	600–1023px	Content max-width ~600-700px centred; larger touch targets retained; admin summary cards in a 2-column grid.
lg+ (desktop)	≥ 1024px	Content max-width ~960px centred with side padding; admin summary cards in a 4-column grid; menu grid shown at full size without scrolling.

13. Authentication & Security Requirements

Security is a first-class requirement. The following MUST all be implemented — this is not optional hardening.

13.1 PIN Storage & Verification

- PINs are never stored or transmitted in plaintext at rest. Only bcrypt hashes (cost factor 10) are stored in Users.pin_hash.
- apps/backend/scripts/generatePinHash.ts is a small CLI (node generatePinHash.ts <pin>) that prints a bcrypt hash for use in seed.sql / environment configuration.
- Login compares the submitted PIN against the stored hash using bcrypt.compare — never simple string equality.

13.2 JWT Sessions

- On successful login, sign a JWT with jsonwebtoken using a secret from process.env.JWT_SECRET, payload { sub: userId, role, displayName }, expiry 12h (process.env.JWT_EXPIRY, default "12h").
- authMiddleware.ts verifies the token on every protected route (Authorization: Bearer <token>), attaches req.user, and returns 401 on missing/invalid/expired tokens.
- A separate requireRole('admin') middleware guards /api/admin/* routes and returns 403 for non-admin tokens.

13.3 Express Middleware Stack (apps/backend/src/app.ts)

Apply in this order:

33. helmet() — sets secure HTTP headers (Content-Security-Policy, X-Content-Type-Options, X-Frame-Options: DENY, Strict-Transport-Security, etc.). Configure CSP to allow only "self" for scripts/styles plus the Azure SWA origin.
34. cors({ origin: process.env.ALLOWED_ORIGIN, credentials: true }) — restrict to the deployed frontend origin only; do not use a wildcard "*" in production.
35. express.json({ limit: '50kb' }) — small body size limit appropriate for order payloads.
36. express-rate-limit on /api/auth/login: windowMs 15 * 60 * 1000, max 5 requests per IP, with a clear 429 error message.
37. Request logging middleware (e.g. morgan in "combined" format to stdout for Azure Log Stream) — MUST NOT log request bodies (to avoid logging PINs) or Authorization headers.
38. Centralised errorHandler.ts as the final middleware — catches thrown errors, returns { error: "..." } with the correct status code, and in production NEVER includes stack traces or internal error details in the response body (log them server-side only).

13.4 SQL Injection Prevention

- ALL database access goes through apps/backend/src/db/pool.ts, a singleton ConnectionPool from the "mssql" package, configured with encrypt: true.
- Every query MUST use request.input('paramName', sql.Type, value) placeholders — string concatenation or template-literal interpolation into SQL text is strictly forbidden anywhere in the codebase. Example:

```
const request = pool.request();
request.input('orderDate', sql.Date, orderDate);
const result = await request.query(
  'SELECT * FROM Orders WHERE order_date = @orderDate ORDER BY id DESC'
);
```

13.5 Input Validation (zod)

- Every request body and relevant query parameter is validated with a zod schema in apps/backend/src/validators/ before reaching the controller logic. Invalid input returns 400 with a descriptive (but non-sensitive) error message.
- Date query parameters are validated against a strict YYYY-MM-DD regex/zod date schema before being passed to SQL.

13.6 XSS Prevention

- React escapes all rendered text by default — dangerouslySetInnerHTML MUST NOT be used anywhere in the codebase.

- Helmet's CSP header (13.3) further mitigates injected-script risks.
- All user-influenced strings (currently none beyond fixed menu item names and badges — there is no free-text customer name field) are still passed through React's normal text rendering, never HTML-injected.

13.7 Transport Security

- Azure Static Web Apps and Azure Functions provide HTTPS by default with automatically managed certificates — HTTP requests are redirected to HTTPS. No manual certificate management is required.

13.8 Secrets Management

- No secrets (JWT secret, DB credentials, PIN hashes) are ever committed to the repository. apps/backend/local.settings.example.json documents the required keys with placeholder values; the real local.settings.json is gitignored.
- In production, all secrets are configured as Application Settings on the Azure Static Web App / Functions resource (see Section 16).

14. Testing Requirements

14.1 Backend (Jest + Supertest + ts-jest)

Location: apps/backend/tests/. Run via npm run test in apps/backend.

Test file	Coverage
pricing.test.ts	buildMenu() returns exactly 24 unique items whose fullPrice/halfPrice exactly match Sections 9.1 and 9.2; computeHalfPrice() formula matches the worked examples in Section 9.3.
auth.test.ts	POST /api/auth/login: correct PIN → 200 + valid JWT; wrong PIN → 401; missing fields → 400; rate limiting after 5 failed attempts → 429.
menu.test.ts	GET /api/menu requires auth (401 without token); returns 24 items with correct fullPrice/halfPrice values.
orders.test.ts	POST /api/orders: creates order + items, server-recomputed totals match Section 10.2 examples; rejects empty items array (400); rejects unknown menuItemId (400). GET /api/orders?date=: returns correct shape and only orders for the given date. PATCH /.../complete: completes a non-pending order with empty body; rejects completing a pending order without paymentMethod (400); completes and updates payment_method when provided. DELETE /api/orders/:id: deletes order + cascades items; 404 for unknown id.
admin.test.ts	GET /api/admin/summary requires admin role (403 for staff token); aggregates totals, cash/upi splits, pending amount, and itemBreakdown sorted by quantity correctly for a seeded set of orders.

- Use a test database (a separate Azure SQL database or a local SQL Server/SQLite-compatible mock via an in-memory mssql mock) seeded with the schema and a small fixed dataset before each test run; clean up after.
- Target: ≥80% statement coverage on src/services and src/utils.

14.2 Frontend (Vitest + React Testing Library)

Location: apps/frontend/tests/. Run via npm run test in apps/frontend.

Test file	Coverage
pricing.test.ts	Menu items match the fixed fullPrice/halfPrice grids in Sections 9.1 and 9.2; order total calculation sums multiple lines correctly including mixed full/half items.
PinPad.test.tsx	Renders 4 empty dots; typing 4 digits fills all dots and triggers onComplete; wrong PIN triggers shake/error state and resets.
MenuGrid.test.tsx	Renders 24 cells with correct display names; clicking a cell increments quantity and shows the count badge; long-press/right-click toggles half-plate and shows the "½ plate" tag.
TotalBar.test.tsx	Displays "₹0" and a disabled submit button with no items; updates total and enables submit when items are added.
OrderCard.test.tsx	Renders correct badges for each orderType/paymentMethod combination; renders item list with quantities and "(½)" suffix for half items; shows "✓ Done" status and reduced styling when completed.
PaymentModal.test.tsx	Renders only when an order with paymentMethod "pending" is marked complete; Cash/UPI buttons call onResolve with the correct value; Cancel dismisses without calling onResolve.

- Mock all API calls with msw (Mock Service Worker) or vitest mocks — no real network calls in unit tests.
- Provide npm run test (root) that runs both workspaces' test suites via npm workspaces (npm run test --workspaces).

15. Deployment & DevOps Plan

15.1 Azure Resources Required

Resource	Tier / Plan	Notes
Resource Group	N/A	Single resource group, e.g. rg-millets-momo
Azure SQL logical server	N/A (server is free; only the database is billed)	Allow Azure services to access server (firewall rule)
Azure SQL Database	Free offer tier (serverless, "Free" selected explicitly at creation)	CRITICAL: must explicitly pick the Free offer — it is not the default. 100,000 vCore-seconds + 32GB storage/month, free indefinitely.
Azure Static Web App	Free plan	Hosts the built frontend AND a managed Azure Functions app for /api/* (built from apps/backend)

15.2 Step-by-Step Setup (for documentation in docs/DEPLOYMENT.md)

39. Create a resource group: az group create --name rg-millets-momo --location centralindia
40. Create the SQL logical server: az sql server create --name <server-name> --resource-group rg-millets-momo --location centralindia --admin-user <admin> --admin-password <strong-password>
41. Allow Azure services to connect: az sql server firewall-rule create --resource-group rg-millets-momo --server <server-name> --name AllowAzureServices --start-ip-address 0.0.0.0 --end-ip-address 0.0.0.0
42. Create the database via the Azure Portal (recommended over CLI for this step) and on the "Compute + storage" page, explicitly select the "Free offer" pricing tier — confirm the page shows "Free" before creating.
43. Run apps/backend/src/db/schema.sql and seed.sql against the new database (via npm run db:migrate using scripts/migrate.ts, or the Azure Portal Query Editor) to create tables and seed the menu + users.
44. Create the Static Web App, linking it to the GitHub repository, with: app location = apps/frontend, output location = dist, api location = apps/backend (Azure auto-detects the Functions app).
45. In the Static Web App's configuration ("Environment variables"/Application Settings), set the backend secrets: SQL_SERVER, SQL_DATABASE, SQL_USER, SQL_PASSWORD, JWT_SECRET, ALLOWED_ORIGIN (the SWA's default *.azurestaticapps.net domain).
46. Push to the configured branch — GitHub Actions (azure-deploy.yml, auto-generated by the SWA setup) builds and deploys both the frontend and the API automatically.
47. Verify: open the SWA URL on a phone, confirm HTTPS, log in as Staff and Admin, place a test order, and confirm "Add to Home Screen" (iOS) / "Install app" (Android) is offered.

15.3 GitHub Actions Workflow

.github/workflows/azure-deploy.yml uses the official Azure/static-web-apps-deploy action, with app_location: "apps/frontend", api_location: "apps/backend", output_location: "dist", and the deployment token stored as a GitHub repository secret (AZURE_STATIC_WEB_APPS_API_TOKEN). On pull requests, the action creates a temporary staging environment; on merge to main, it deploys to production.

15.4 Database Migration Script

apps/backend/scripts/migrate.ts connects using the same env vars as the runtime, reads schema.sql and seed.sql, and executes them in order (idempotent — uses IF NOT EXISTS / checks before CREATE where appropriate). Exposed as npm run db:migrate.

16. Environment Variables & Configuration

16.1 Backend (apps/backend/local.settings.example.json)

```
{
  "IsEncrypted": false,
  "Values": {
    "FUNCTIONS_WORKER_RUNTIME": "node",
    "AzureWebJobsStorage": "",
    "NODE_ENV": "development",

    "SQL_SERVER": "<server-name>.database.windows.net",
    "SQL_DATABASE": "millets-momo-db",
```

```
    "SQL_USER": "<admin-username>",
    "SQL_PASSWORD": "<admin-password>",
    "SQL_ENCRYPT": "true",

    "JWT_SECRET": "<random-long-secret-string>",
    "JWT_EXPIRY": "12h",

    "ALLOWED_ORIGIN": "http://localhost:5173"
  }
}
```

16.2 Frontend (apps/frontend/.env.example)

```
VITE_API_BASE_URL=http://localhost:7071/api
```

16.3 Variable Reference Table

Variable	Where	Description
SQL_SERVER / SQL_DATABASE / SQL_USER / SQL_PASSWORD	Backend	Azure SQL connection details (Section 8)
SQL_ENCRYPT	Backend	Must be "true" — Azure SQL requires encrypted connections
JWT_SECRET	Backend	Signing secret for session JWTs (Section 13.2)
JWT_EXPIRY	Backend	Token lifetime, default "12h"
ALLOWED_ORIGIN	Backend	CORS allow-list — the deployed frontend URL
VITE_API_BASE_URL	Frontend	Base URL for API calls; same-origin "/api" in production behind SWA

17. README Requirements

The root README.md MUST contain the following sections, in this order:

48. Project title, one-paragraph description, and a "Built with" badge-style list (React, MUI, Node.js, Azure).
49. Features — a bullet list mirroring Section 2.1.
50. Tech stack table (reuse Section 4).
51. Monorepo structure — an abbreviated version of the tree in Section 5.
52. Prerequisites — Node.js 20+, npm, an Azure SQL Database (or local SQL Server) for development.
53. Local setup — step-by-step: clone repo, npm install (root, installs all workspaces), copy .env.example / local.settings.example.json files and fill in values, run npm run db:migrate, run npm run dev (concurrently starts frontend Vite dev server and backend Functions host).
54. Default accounts — explains that Staff and Admin PINs are set via the seed/migration step (Section 9.5 / 16) and how to generate new PIN hashes with scripts/generatePinHash.ts.
55. Build & test — npm run build, npm run test, npm run lint commands for the whole monorepo.

56. Deployment — summary + link to docs/DEPLOYMENT.md (Section 15).
57. PWA installation — short instructions for installing on iOS Safari and Android Chrome.
58. License — MIT (or as the business owner prefers; default to "Proprietary — internal use only" given this is a private business tool).

18. Non-Functional Requirements

18.1 Performance

- First meaningful paint < 2 seconds on a typical 4G connection (Azure SWA + small bundle should comfortably achieve this).
- API p95 response time < 300ms for all endpoints under the expected load (≤10 concurrent users, low request volume).
- Frontend main JS bundle (gzipped) target < 300KB; use code-splitting (React.lazy) for the Admin Dashboard route since Staff users never load it.

18.2 Scalability / Concurrency

- Designed for ≤10 concurrent users. Azure SQL Free offer's 100,000 vCore-seconds/month and Azure Functions' 1M free executions/month both provide enormous headroom at this scale — no additional scaling configuration is required.

18.3 Accessibility

- All interactive elements (PIN pad keys, menu cells, buttons) have a minimum touch target of 44×44px.
- Colour contrast for text on coloured badges/backgrounds meets WCAG AA (verify the palette in Section 6.1 against badge text colours).
- All icon-only buttons (e.g. delete "✕") have an aria-label.

18.4 Code Quality

- TypeScript strict mode enabled in both apps/frontend/tsconfig.json and apps/backend/tsconfig.json.
- ESLint + Prettier configured at the root and inherited by both workspaces; npm run lint must pass with zero errors.
- Every exported function/component has a JSDoc/TSDoc comment describing its purpose, parameters, and return value.
- No unused dependencies; keep the dependency list minimal (justify any addition beyond Section 4's stack).

18.5 Browser & Device Support

- Latest two versions of Chrome (Android & desktop) and Safari (iOS & macOS), plus Edge.
- Verified installable as a PWA on iOS 16+ Safari and Android 12+ Chrome.

19. Acceptance Criteria / Definition of Done

The implementation is considered complete when ALL of the following are true:

19.1 Functional

59. All 15 functional requirements in Section 2.1 are implemented and match the prototype's design and copy.
60. Staff can log in, select a date, create an order with any combination of the 24 menu items (full or half plate), set order type and payment method, and submit it; the order appears immediately in the Day View.
61. Marking a "Cash"/"UPI" order complete moves it to the Completed section with no extra prompts.
62. Marking a "Pending" order complete shows the PaymentModal, requires Cash or UPI selection, updates the stored payment method, and then moves the order to Completed.
63. Deleting an order requires confirmation and permanently removes it (and its items) from the database.
64. Admin can log in separately, select any date, and see accurate summary cards and item breakdown that match the underlying order data for that date (verified by at least one integration test comparing API output to seeded data).

19.2 Design

65. The MUI theme in Section 6 is implemented exactly — colours, radii, shadows, and the status badge tokens are used consistently across all screens.
66. All animations listed in Section 6.7 are present and feel smooth (no jank) on a mid-range mobile device.
67. Layouts adapt correctly at the breakpoints in Section 12.4, verified by manual resize testing and/or responsive screenshots.

19.3 PWA

68. Lighthouse PWA audit score ≥ 90.
69. App is installable via "Add to Home Screen" on iOS Safari and "Install app" on Android Chrome, launches in standalone mode with the correct icon, name, and theme colour.
70. App shell loads when offline (airplane mode) after first visit.

19.4 Security

71. PINs are bcrypt-hashed; no plaintext PINs exist anywhere in code, config, or logs.
72. All protected endpoints reject requests without a valid JWT (401) and admin-only endpoints reject non-admin tokens (403).
73. Helmet, CORS allow-listing, rate limiting on login, and the global error handler are all active and verified by tests.
74. Every SQL query uses parameterized inputs — a manual code review/grep confirms no string-concatenated SQL exists.
75. zod validation rejects malformed requests (400) for every endpoint that accepts input.

19.5 Testing & Documentation

76. npm run test (root) passes for both workspaces with the coverage targets in Section 14.
77. npm run lint and npm run build succeed for both workspaces with zero errors.

78. README.md contains all sections listed in Section 17 and a new developer can follow it to run the app locally without external help.
79. docs/ARCHITECTURE.md, docs/API.md, and docs/DEPLOYMENT.md exist and accurately describe the implemented system.

19.6 Deployment

80. The app is successfully deployed to Azure Static Web Apps with the Azure SQL Database on the Free offer tier, confirmed reachable over HTTPS from a mobile browser, with all environment variables configured per Section 16 (no secrets committed to the repo).

20. Implementation Roadmap (Suggested Build Order)

The coding agent should implement the project in the following phases. Each phase should result in a working, committable state.

1. Phase 1 — Monorepo scaffolding: root package.json with npm workspaces, .gitignore, .editorconfig, ESLint/Prettier configs, empty apps/frontend and apps/backend Vite/Functions projects, packages/shared with menu.ts and pricing.ts (Section 9).
2. Phase 2 — Theme: implement apps/frontend/src/theme/theme.ts and tokens.ts exactly per Section 6; create a minimal App.tsx that renders a themed placeholder page to verify the theme visually.
3. Phase 3 — Database: write schema.sql and seed.sql (Sections 8 & 9.4), scripts/generatePinHash.ts and scripts/migrate.ts, and apps/backend/src/db/pool.ts.
4. Phase 4 — Backend auth: authService, authController, authRoutes, authMiddleware, rateLimiter, and the Express app skeleton (app.ts) with helmet/cors/json/error-handler wired in (Section 13); functions/api.ts wraps it for Azure Functions. Write auth.test.ts.
5. Phase 5 — Backend menu & orders: menuController/Service, ordersController/Service (with server-side total recomputation per Section 10.2), adminController/Service. Write menu.test.ts, orders.test.ts, admin.test.ts, pricing.test.ts.
6. Phase 6 — Frontend auth flow: AuthContext, axios client with interceptors, LoginPage with PinPad, ProtectedRoute, routing skeleton in App.tsx. Write PinPad.test.tsx.
7. Phase 7 — Frontend Staff flow: DateSelectPage, DayViewPage (OrderCard, StatChip), NewOrderPage (MenuGrid, OrderConfigPanel, SelectedItemsList, TotalBar), OrderDraftContext, react-query hooks. Write MenuGrid.test.tsx, TotalBar.test.tsx, OrderCard.test.tsx, pricing.test.ts (frontend copy).
8. Phase 8 — Pending-payment completion flow: PaymentModal component and integration with OrderCard/DayViewPage per Section 10.4. Write PaymentModal.test.tsx.
9. Phase 9 — Admin dashboard: AdminDashboardPage with summary cards, item breakdown table, and orders list, using useAdminSummary.
10. Phase 10 — PWA: configure vite-plugin-pwa, manifest, icons (generate a simple momo-themed icon set at the required sizes), iOS meta tags (Section 7). Run and fix Lighthouse PWA audit.
11. Phase 11 — Polish: animations (Section 6.7), responsive QA across breakpoints (Section 12.4), validation messages (Section 10.7), toasts.
12. Phase 12 — Documentation: README.md (Section 17), docs/ARCHITECTURE.md, docs/API.md (derived from Section 11), docs/DEPLOYMENT.md (Section 15).

13. Phase 13 — Deployment: provision Azure resources per Section 15, configure GitHub Actions, set environment variables, run migrations against the live database, and verify PWA installability on real iOS and Android devices.

— End of PRD —